FILE ENCRYPTION AND DECRYPTION USING AES ALGORITHM

A SUMMER INTERNSHIP REPORT

Submitted by

KAVINRAJAN K– 810023205113



DEPARTMENT OF INFORMATION TECHNOLOGY

ANNA UNIVERSITY

TIRUCHIRAPALLI – 620 024.

UNIVERSITY COLLEGE OF ENGINEERING(BIT CAMPUS),

ANNA UNIVERSITY , TIRUCHIRAPALLI – 620 024

DEPARTMENT OF INFORMATION TECHNOLOGY



BONAFIDE CERTIFICATE

This is to certify this Summer Internship project report **"File Encryption and Decryption using AES Algorithm"** is the Bonafide work of **KAVINRAJAN K (810023205113)** has satisfactory completed the course of **IT3711 – SUMMER INTERNSHIP** for **SEVENTH** semester, **B.TECH INFORMATION TECHNOLOGY** during the academic year **2026-2027.**

Faculty Incharge **Head of the Department**

Submitted for the practical examination held on _____.

ABSTRACT

The File Encryption and Decryption System Using AES Algorithm is a cybersecurity-based application developed to provide a secure and reliable method for protecting sensitive digital files from unauthorized access. The system enables users to upload files, encrypt them using the Advanced Encryption Standard (AES) algorithm, securely store the encrypted files, and decrypt them only with the correct encryption key. This ensures the confidentiality and integrity of important data while reducing the risk of data theft, tampering, or unauthorized disclosure.

The project is implemented using Java with Spring Boot for the backend, providing a scalable, secure, and maintainable architecture for handling business logic, user authentication, file processing, and RESTful web services. Hibernate/JPA is used for database persistence to manage user information and encrypted file records. The frontend is developed using HTML, CSS, JavaScript, and Bootstrap, offering a responsive and userfriendly interface for file upload, encryption, decryption, and download. This combination of technologies ensures efficient performance and ease of use for both administrators and end users.

The primary objective of this system is to enhance data security by replacing traditional file storage methods with strong cryptographic protection. The application minimizes the chances of unauthorized access by encrypting files before storage and allowing decryption only after successful user authentication and key verification. It also incorporates rolebased access control, secure key management, file handling, and audit capabilities to improve the overall security of the application. By automating the encryption and decryption process, the system reduces manual effort while ensuring the confidentiality of sensitive information.

Overall, the File Encryption and Decryption System Using AES Algorithm demonstrates the practical implementation of modern cybersecurity principles and secure software development practices learned during the internship. It showcases the integration of a robust Java Spring Boot backend with an interactive web interface to build a complete file security solution. The project serves as an effective example of how cryptographic techniques, particularly the AES algorithm, can be applied to protect digital data and strengthen information security in real-world applications.

CONTENTS

CHAPTER NO	TABLE OF CONTENTS	PAGE NO
1	INTRODUCTION 1.1 OBJECTIVE OF THE INTERNSHIP 1.2 OBJECTIVE OF THE FILE ENCRYPTION AND DECRYPTION USING AES ALGORITHM	05
2	COMPANY PROFILE	06
3	DAILY WORK REPORT	07-09
4	BLOCK DIAGRAM	10
5	FILE ENCRYPTION AND DECRYPTION SYSTEM 5.1. EXCITING SOLUTION 5.2 PROPOSED SOLUTION 5.3 CAPABILITIES	11
6	PROPOSED METHODS 6.1 Role-Based Access Control (RBAC) 6.2 File Encryption Module 6.3 File Decryption Module 6.4 Secure File Storage Management 6.5 AES key Management 6.6 File access Monitoring	12-14
7	ALGORITHMS	15
8	CODE IMPLEMENTATION	16-24
9	SAMPLE OUTPUT	25-26

10	ANALYSIS	27
11	CONCLUSION	28
12	FUTURE WORK	28
13	REFERENCES	29

1. INTRODUCTION

The **File Encryption and Decryption System Using AES Algorithm** is a cybersecurity-based application developed to protect sensitive digital files from unauthorized access and data breaches. In today's digital world, users store confidential information such as personal documents, financial records, business reports, and other important files on computers and cloud platforms. Without proper security, these files are vulnerable to cyber threats, including hacking, malware, and unauthorized access.

To overcome these security challenges, this project provides a secure web-based solution that allows users to encrypt their files using the **Advanced Encryption Standard (AES)** algorithm before storing or sharing them. The encrypted files can only be decrypted by authorized users possessing the correct encryption key, ensuring confidentiality and data protection. The application also provides secure user authentication, file upload, encryption, decryption, and download functionalities through an easy-to-use interface.

The project is developed using **Java, Spring Boot, Hibernate/JPA, MySQL, HTML, CSS, JavaScript, and Bootstrap**, enabling users to securely manage their confidential files while applying modern cybersecurity concepts and software development practices learned during the internship.

1.1 Objective of the Internship

- To gain practical knowledge of **Cyber Security** concepts and secure software development practices.
- To understand cryptography techniques and implement secure data protection methods using the **AES encryption algorithm**.
- To enhance programming skills using **Java, Spring Boot, Hibernate/JPA, MySQL, HTML, CSS, JavaScript, and Bootstrap**.
- To gain hands-on experience in designing, developing, testing, and deploying a secure web application in a professional environment.
- To improve problem-solving, debugging, teamwork, and documentation skills through real-time project development during the internship.

1.2 Objective of the File Encryption and Decryption System Using AES Algorithm

- To design and develop a secure web application for encrypting and decrypting files using the **Advanced Encryption Standard (AES)** algorithm.
- To protect sensitive files from unauthorized access by converting plaintext files into encrypted ciphertext.
- To provide secure user authentication and authorization before allowing file encryption or decryption.

2.COMPANY PROFILE

Fantasy Solution:

Trichy is a leading IT solutions and service provider offering software development, web designing, web hosting, and IT training services.

The organization specializes in real-time project development, application development & maintenance, ERP implementation & consulting, testing, performance engineering, and software localization.

Fantasy Solution has contributed to the formulation of several national IT standards and continues to deliver technology solutions in fields such as automotive electronics, smart devices, digital home products, and enterprise applications.

Address:
Fantasy Solution,
No 16, Samnath Plazza,
Third Floor, Near
BG Naidu Sweets,
Melapudur, Trichy
– 1.
Contact: 9043095535
Landline: 0431-4971630 Website: www.fantasysolution.in

Mission:

Fantasy Solution is committed to providing high quality software development services, secure access to information, improved communication between businesses, value-formoney solutions to customers, and meaningful growth opportunities for its employees.

Vision:
To be a leading software development and training organization by delivering innovative, high-quality, and industry-oriented technology solutions. The company aims to empower students and organizations through focused project guidance, practical learning, and professional software development services that enhance technical skills and support career growth. It strives to bridge the gap between academic learning and industry requirements by providing real-time project experience and continuous technological innovation.

3. DAILY WORK REPORT:

Domain: Cyber Security
Project: File Encryption and Decryption System Using AES Algorithm
Company: Fantasy IT Solutions

Day 1
On the first day of my internship, I attended the orientation program conducted by Fantasy IT Solutions. I was introduced to the company's work environment, cybersecurity domain, and internship guidelines. The project titled **"File Encryption and Decryption System Using AES Algorithm"** was assigned, and its objectives and scope were explained.

Day 2
I learned the basic concepts of Cyber Security, including the CIA Triad (Confidentiality, Integrity, and Availability). The importance of securing digital information and protecting systems from cyber threats was discussed. This session provided a strong foundation for understanding the project.

Day 3
I studied different types of cyber attacks such as phishing, malware, ransomware, brute-force attacks, and social engineering. The mentor explained how these attacks affect organizations and the methods used to prevent them. I also learned the importance of secure file storage.

Day 4
The focus was on cryptography and data protection techniques. I learned about symmetric and asymmetric encryption algorithms, hashing, and digital signatures. Different encryption standards such as AES, DES, and RSA were compared.

Day 5
I studied the AES encryption algorithm in detail, including its working process, key sizes, and security features. The mentor explained why AES is widely used for protecting confidential files and sensitive information.

Day 6
The project requirements were analyzed, and the Software Requirement Specification (SRS) was prepared. The functional and non-functional requirements of the application were identified, and the project modules were finalized.

Day 7

I designed the overall system architecture and block diagram for the application. The workflow of file upload, encryption, decryption, and secure download was discussed and documented.

Day 8

The development environment was configured by installing Java, Spring Boot, Maven, MySQL, and the required IDE. The project structure was created, and database connectivity was established successfully.

Day 9

The database schema was designed, and tables for users, encrypted files, and activity logs were created. Proper relationships between the tables were established to ensure efficient data management.

Day 10

I developed the user registration module with proper input validation. Password encryption and secure data storage methods were implemented to improve application security.

Day 11

The user login module was developed using authentication techniques. Access control mechanisms were implemented to ensure that only authorized users could access the application.

Day 12

The file upload module was implemented. The application was designed to validate uploaded files before processing them for encryption.

Day 13

The AES encryption algorithm was integrated into the application. Uploaded files were successfully encrypted using a secure secret key before being stored.

Day 14

I implemented the file decryption module. The system was able to decrypt encrypted files only after verifying the user's authentication and the correct encryption key.

Day 15

Secure file download functionality was added. Users could download only their own decrypted files after successful verification.

Day 16

Exception handling and input validation were improved throughout the application. This increased the reliability and security of the system.

Day 17

The frontend pages were designed using HTML, CSS, JavaScript, and Bootstrap. A simple and user-friendly interface was developed for file encryption and decryption.

Day 18

The frontend was integrated with the Spring Boot backend using REST APIs. Data communication between the client and server was tested successfully.

Day 19

Unit testing was performed for the registration, login, encryption, and decryption modules. The outputs were verified with different test cases.

Day 20

Integration testing was conducted to ensure that all modules worked together correctly. Minor issues were identified and resolved.

Day 21

Additional security features such as secure key validation and file access control were implemented. The application was made more secure against unauthorized access.

Day 22

Application performance was analyzed by testing encryption and decryption of different file types. The processing speed and accuracy were verified.

Day 23

Several bugs identified during testing were fixed. The application performance and user experience were improved through code optimization.

Day 24

Project documentation was updated with system design, module descriptions, and implementation details. Screenshots of each module were also collected.

Day 25

The block diagram, workflow diagram, and algorithm explanations were prepared for the internship report. The project documentation was reviewed by the mentor.

Day 26

The application was tested with multiple file formats such as PDF, Word documents, images, and text files. The encryption and decryption process worked successfully.

Day 27

The final code was reviewed and optimized. Security features and error handling mechanisms were verified to ensure stable application performance.

Day 28

The sample outputs and project results were documented. The project objectives were compared with the achieved outcomes, and the implementation was evaluated.

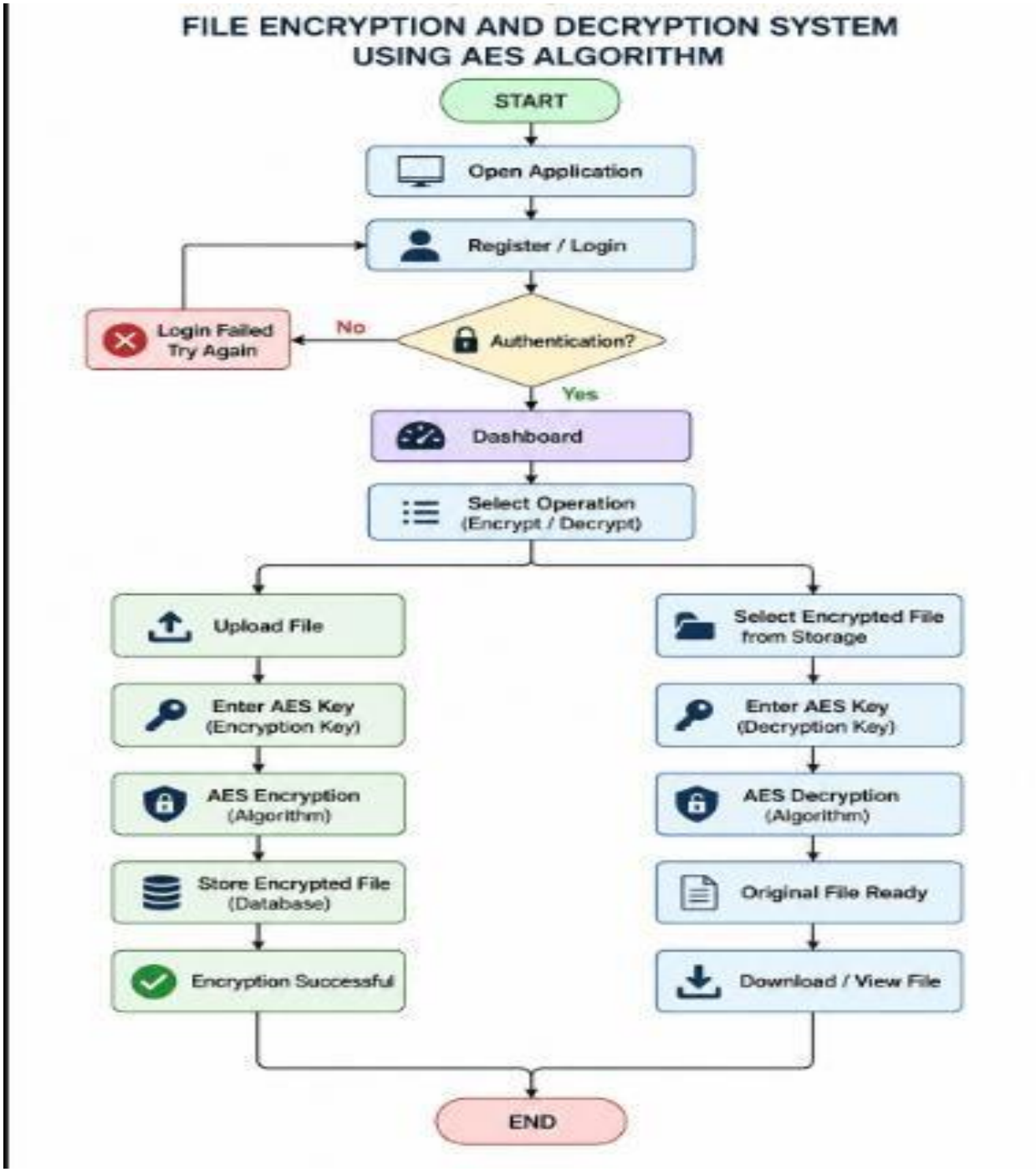
Day 29

The internship report was completed by preparing the abstract, methodology, analysis, conclusion, and future enhancements. All required documents were finalized.

Day 30

On the final day, I presented the **File Encryption and Decryption System Using AES Algorithm** to my mentor. The complete project was demonstrated successfully, and feedback was received. This internship enhanced my practical knowledge of cybersecurity, cryptography, secure software development, and full-stack application development.

4.BLOCK DIAGRAM:



5. FILE ENCRYPTION AND DECRYPTION SYSTEM

The File Encryption and Decryption System Using AES Algorithm is designed to provide a secure platform for protecting sensitive digital files from unauthorized access. The system allows users to upload files, encrypt them using the Advanced Encryption Standard (AES) algorithm, store them securely, and decrypt them only with the correct secret key. This ensures that confidential information remains protected during storage and transmission. The application also includes user authentication, secure file management, and access control to improve overall data security.

5.1 EXISTING SOLUTION

In the existing system, users often store important files without encryption or use weak security mechanisms such as password-protected folders or simple compression tools. These methods do not provide sufficient protection against cyber threats, making confidential files vulnerable to unauthorized access, theft, and modification. Many existing systems also lack proper user authentication, secure key management, and controlled access to sensitive files.

Limitations of the Existing System

- Files are stored in plain text without strong encryption.
- High risk of unauthorized access and data theft.
- Weak or no encryption algorithms are used.
- Lack of secure key management.
- No proper authentication or access control.
- Increased chances of data tampering and security breaches.

5.2 PROPOSED SOLUTION

The proposed system introduces a File Encryption and Decryption System using the AES (Advanced Encryption Standard) algorithm to provide strong security for confidential files. Users must first register and log in before accessing the system. Files uploaded by the user are encrypted using a secret AES key before being stored in the database or storage location. Whenever a user wants to access the file, the encrypted file is retrieved and decrypted only after successful authentication and verification of the correct encryption key. This ensures that only authorized users can view or download the original file.

Advantages of the Proposed System

- Strong file protection using AES encryption.
- Secure user authentication and authorization.
- Confidentiality of sensitive information is maintained.

- Secure storage of encrypted files.
- Easy encryption and decryption through a user-friendly interface.
- Reduces the risk of data leakage and cyber attacks.

5.3 CAPABILITIES

The File Encryption and Decryption System Using AES Algorithm provides the following capabilities:

- Secure user registration and login.
- File upload with input validation.
- AES-based file encryption using a secret key.
- Secure storage of encrypted files.
- File decryption only with the correct encryption key.
- Download and view decrypted files securely.
- Protection against unauthorized file access.
- Fast and reliable encryption and decryption process.
- User-friendly interface for easy file management.
- Improved confidentiality, integrity, and security of digital data.

6. PROPOSED METHOD

File Encryption and Decryption System Using AES Algorithm

The proposed File Encryption and Decryption System Using AES Algorithm is designed to provide secure storage and transmission of confidential files. The system uses the Advanced Encryption Standard (AES) algorithm to encrypt files before storing them and decrypt files only for authorized users. This prevents unauthorized access and protects sensitive information from data breaches.

The system provides a secure environment where users can upload files, encrypt them using AES, store encrypted files, and retrieve the original files through the decryption process using a valid secret key.

6.1 Role-Based Access Control (RBAC)

The proposed system implements Role-Based Access Control to restrict file access based on user roles and permissions. Each user is assigned a specific role, and access to encryption and decryption operations is provided according to their authorization level.

Features:

- Users must authenticate before accessing the system.
- Different users are provided with different access permissions.
- Only authorized users can encrypt, decrypt, upload, or download files.
- Prevents unauthorized users from accessing confidential files.
- Maintains secure control over sensitive data.

Roles:

- Administrator: Manages users, permissions, and overall system security.
- Authorized User: Uploads files, performs encryption, and accesses permitted files.
- Security Manager: Monitors encryption activities and file access logs.

6.2 File Encryption Module

The file encryption module protects files by converting original files into encrypted data using the AES algorithm.

Features:

- Users can upload files into the system.
- The uploaded file data is converted into plaintext format.
- AES encryption algorithm converts plaintext into ciphertext using a secret key.
- The encrypted file is stored securely in the storage system.
- Original file content is hidden from unauthorized users.

Process:

- User selects and uploads a file.
- The system reads the file data.
- AES algorithm encrypts the file using a secret key.
- The encrypted file is stored securely.
- Only users with the correct key can access the original file.

6.3 File Decryption Module

The file decryption module restores encrypted files back to their original format using the AES decryption process.

Features:

- Allows authorized users to recover original files.
- Uses the same AES secret key for decryption.
- Converts ciphertext back into readable plaintext.
- Ensures secure file retrieval.
- Prevents unauthorized file restoration. Process:
- User requests an encrypted file.
- System verifies user authorization.
- Encrypted data is retrieved from storage.
- AES algorithm decrypts the file using the secret key.
- Original file is provided to the authorized user.

6.4 Secure File Storage Management

The system stores encrypted files instead of storing original files directly. This provides additional security against unauthorized access and data theft.

Features:

- Stores only encrypted versions of files.
- Protects confidential documents from attackers.
- Maintains file details such as file name, size, and upload date.
- Provides secure file retrieval.
- Reduces the risk of data leakage.

6.5 AES Key Management

The AES secret key plays an important role in encryption and decryption operations. The key is used to convert plaintext into ciphertext and recover the original file.

Features:

- Provides secure encryption and decryption operations.
- Validates the key before decrypting files.
- Prevents unauthorized users from accessing encrypted data.
- Maintains confidentiality of stored files.

6.6 File Access Monitoring

The system monitors file-related activities to improve security and maintain user accountability.

Features:

- Records file upload activities.
- Tracks encryption and decryption operations.
- Maintains user access history.
- Helps identify unauthorized access attempts.
- Improves overall system security.

Advantages of Proposed System

- Provides strong file security using AES encryption.
- Prevents unauthorized access to confidential files.
- Ensures secure file storage and transmission.
- Allows controlled access through role-based permissions.
- Maintains data confidentiality and integrity.
- Provides efficient file encryption and decryption.
- Reduces the possibility of data theft and leakage.

Disadvantages of Proposed Solution

File Encryption and Decryption System Using AES Algorithm

1. Key Management Problem
   - The security of AES depends on the secret key.
   - If the encryption key is lost, the original file cannot be decrypted.
   - Secure storage and sharing of keys is challenging.

2. Performance Overhead ○ Encrypting and decrypting large files requires additional processing time and system resources.
   ○ It may reduce performance on low-end devices.
3. No Protection Against Insider Threats ○ If an authorized user has the encryption key, they can access the protected files.

7. ALGORITHM

7.1 Algorithm for File Encryption Using AES Algorithm Process 1: File Upload Steps:

1. The user selects the file that needs to be encrypted.
2. The system receives the uploaded file through the file upload interface.
3. The uploaded file is validated to check the file type and size.
4. The system reads the file content and converts it into a byte stream for encryption.

Process 2: AES Key Generation Steps:

1. The system generates or retrieves the AES secret key.
2. The secret key is used as the encryption key for securing the file.
3. The generated key is securely managed to prevent unauthorized usage.

Process 3: File Encryption Steps:

1. The AES algorithm is initialized in encryption mode.
2. The system provides the AES secret key to the encryption module.
3. The file data is processed using multiple AES encryption rounds.
4. The original file content (plaintext) is converted into encrypted data (ciphertext).
5. The encrypted file data is encoded into a secure format.

Process 4: Secure File Storage Steps:

1. The encrypted file is stored in the secure storage location.
2. The system stores file metadata such as file name, file size, and encryption status.
3. The original file is removed or protected to prevent unauthorized access.
4. The system confirms that the encrypted file is stored successfully.

7.2Algorithm for File Decryption Using AES Algorithm
Process 1: Encrypted File Retrieval Steps:

1. The authorized user requests an encrypted file from the system.
2. The system verifies the user's authentication and access permissions.
3. The encrypted file is retrieved from secure storage.
4. The encrypted file data is loaded into the decryption module.

Process 2: AES Key Verification Steps:

1. The user provides the required AES secret key.
2. The system verifies the provided key with the stored encryption key.

3. If the key is valid, the decryption process continues.
4. If the key is invalid, access to the file is denied.

Process 3: File Decryption Steps:

1. The AES algorithm is initialized in decryption mode.
2. The encrypted file data (ciphertext) is provided to the AES decryption module.
3. The AES algorithm processes the ciphertext using the secret key.
4. The ciphertext is converted back into the original file content (plaintext).
5. The decrypted data is converted into the original file format.

Process 4: File Recovery and Download Steps:

1. The system restores the original file after successful decryption.
2. The decrypted file is made available for authorized users.
3. The system records the decryption activity for security tracking.
4. The user can download or access the recovered original file.
5. The system completes the file decryption process successfully.

8.CODE IMPLEMENTATION:

```
// FileEncryptionApplication.java

package com.example.fileencryption;

import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*; import
org.springframework.http.ResponseEntity; import
org.springframework.data.jpa.repository.JpaRepository; import
org.springframework.stereotype.Repository;

import jakarta.persistence.*;
import org.springframework.beans.factory.annotation.Autowired; import
org.springframework.web.multipart.MultipartFile;

import javax.crypto.Cipher; import
javax.crypto.spec.SecretKeySpec;
import java.io.FileOutputStream; import
java.nio.file.Files; import
```

```
java.nio.file.Paths; import
java.util.Base64;


// ================= MAIN SPRING BOOT APPLICATION ================

@SpringBootApplication public class
FileEncryptionApplication {

   public static void main(String[] args) {

      SpringApplication.run(
            FileEncryptionApplication.class,
args
      );
   }
}


// ================= AES ENCRYPTION MODULE ================

class AESEncryption {

   private static final String SECRET_KEY =
         "1234567890123456";


   public static String encrypt(String data)
throws Exception {


      SecretKeySpec secretKey =
new SecretKeySpec(
            SECRET_KEY.getBytes(),
            "AES");


      Cipher cipher =
            Cipher.getInstance("AES");
```

```
        cipher.init(
                Cipher.ENCRYPT_MODE,
                secretKey);


        byte[] encryptedData =
cipher.doFinal(
data.getBytes());


        return Base64.getEncoder()
                .encodeToString(encryptedData);
    }
}


// ================= AES DECRYPTION MODULE ================

class AESDecryption {

    private static final String SECRET_KEY =
            "1234567890123456";


    public static String decrypt(
String encryptedData)
throws Exception {


        SecretKeySpec secretKey =
new SecretKeySpec(
                SECRET_KEY.getBytes(),
                "AES");


        Cipher cipher =
                Cipher.getInstance("AES");
```

```
        cipher.init(
                Cipher.DECRYPT_MODE,
                secretKey);


        byte[] decryptedData =
cipher.doFinal(
                Base64.getDecoder()
                .decode(encryptedData));


        return new String(decryptedData);
    }
}



// ================= USER ENTITY CLASS =================

@Entity class
User {


    @Id
    @GeneratedValue(strategy =
            GenerationType.IDENTITY)

    private Long id;


    private String username;


    private String password;


    private String email;
```

```
    public Long getId() {
return id;
    }


    public String getUsername() {
return username;
    }


    public void setUsername(String username) {
this.username = username;
    }


    public String getPassword() {
return password;
    }


    public void setPassword(String password) {
this.password = password;
    }


    public String getEmail() {
return email;
    }


    public void setEmail(String email) {
this.email = email;
    }
}



// ================= DATABASE REPOSITORY =================
```

```
@Repository interface UserRepository
extends JpaRepository<User,Long>{


   User findByUsername(String username);

}



// ================== CONTROLLER CLASS ==================

@RestController
@RequestMapping("/api") class
FileEncryptionController {



   @Autowired     private UserRepository
userRepository;



   // -------- USER LOGIN --------

   @PostMapping("/login")
   public ResponseEntity<String> login(
         @RequestBody User user) {


      User existingUser =
userRepository.findByUsername(
user.getUsername());


      if(existingUser != null &&
existingUser.getPassword()
.equals(user.getPassword())) {
```

```
            return ResponseEntity.ok(
                    "Login Successful");

        }


        return ResponseEntity
                .status(401)
                .body(
                "Invalid Login Details");
    }



    // -------- FILE UPLOAD AND ENCRYPTION --------


    @PostMapping("/upload")
public String uploadFile(
            @RequestParam("file")
MultipartFile file)          throws
Exception {



        String fileData =
new String(
file.getBytes());



        String encryptedFile =
AESEncryption.encrypt(
fileData);        FileOutputStream output =
new FileOutputStream(

                "encrypted/"
                +file.getOriginalFilename()
                )
```

```
        output.write(
                encryptedFile.getBytes());


        output.close();



        return "File Encrypted Successfully";
    }

    // -------- FILE DECRYPTION --------


    @GetMapping("/decrypt/{filename}")
public String decryptFile(
@PathVariable String filename)
throws Exception {

        String encryptedData =
                Files.readString(
                Paths.get(
                "encrypted/"
                +filename));

        String originalFile =
AESDecryption.decrypt(
encryptedData);



        return originalFile;
    }




    // -------- DIRECT AES TEST --------
```

```
    @GetMapping("/encrypt/{data}")
public String encryptText(
@PathVariable String data)          throws
Exception {


        return AESEncryption.encrypt(data);

    }



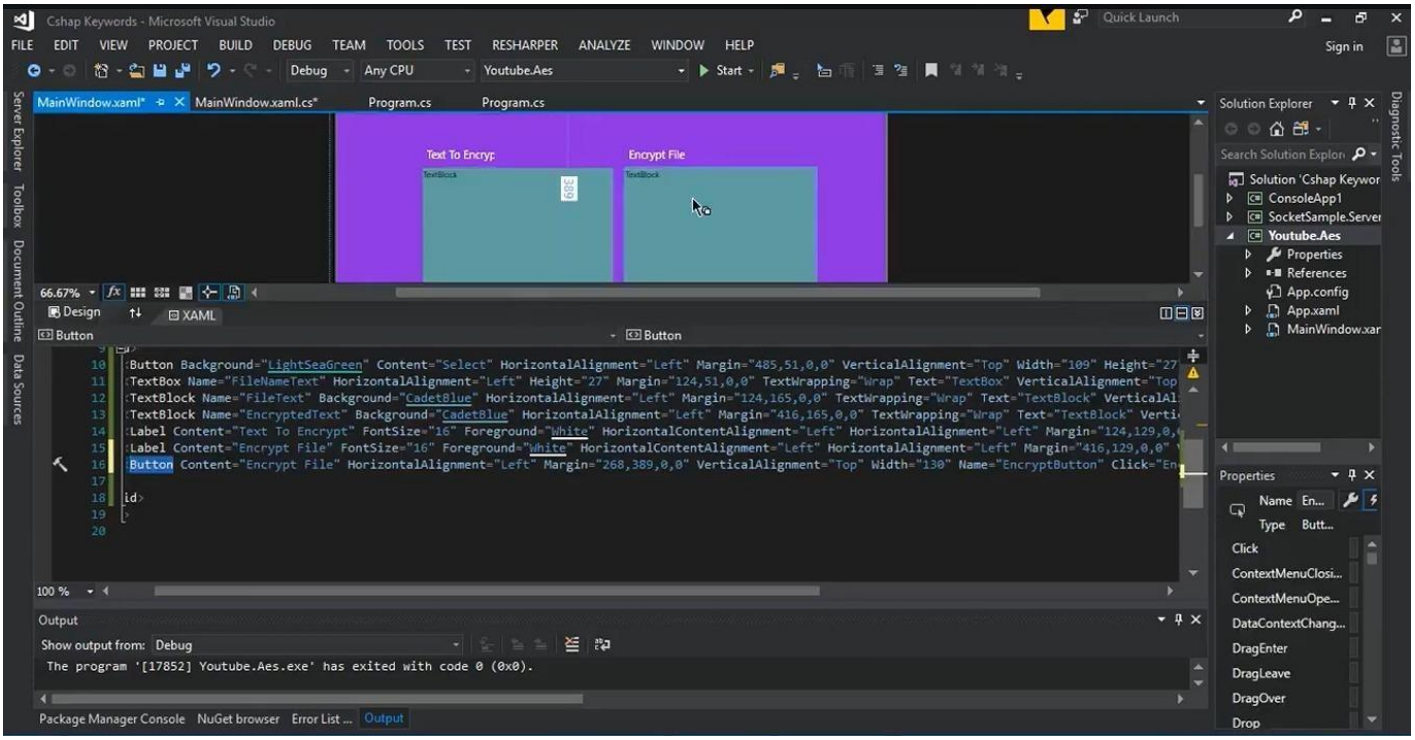
    @GetMapping("/decryptText/{data}")
public String decryptText(
@PathVariable String data)          throws
Exception {


        return AESDecryption.decrypt(data);

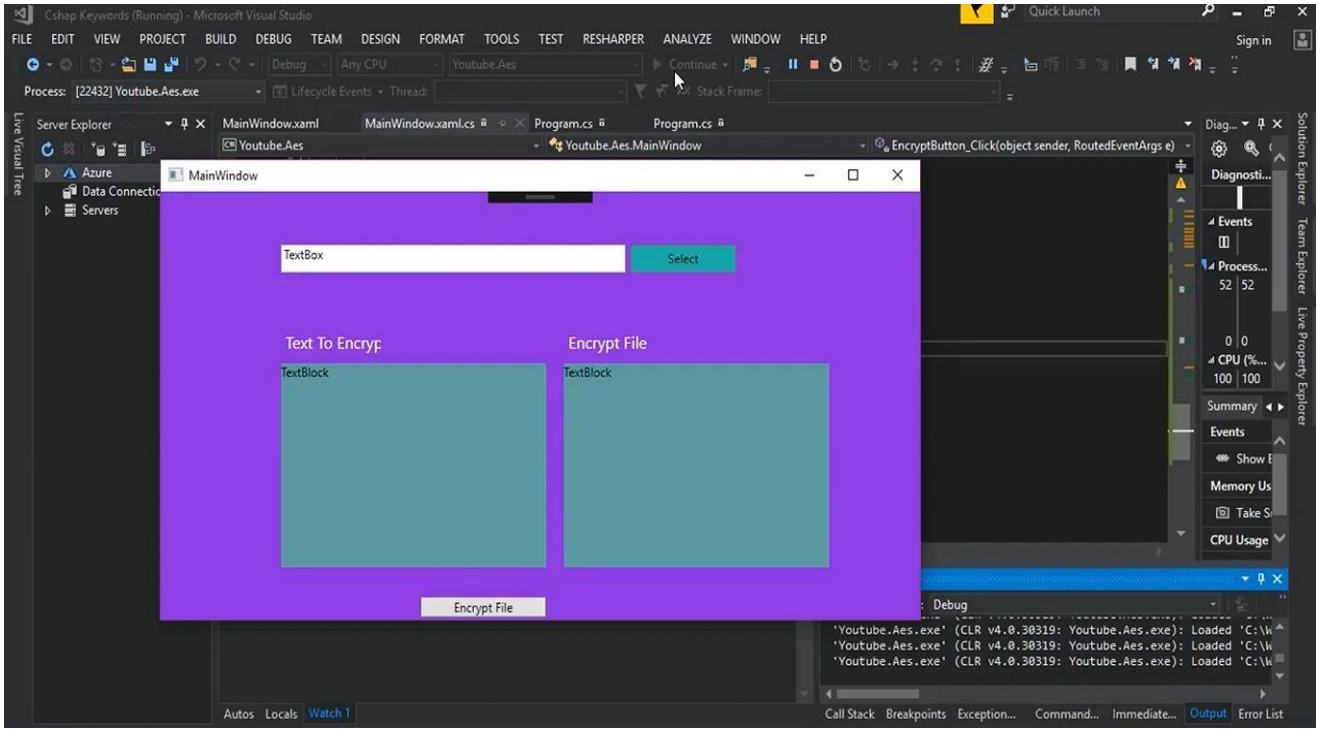
    }

}
```

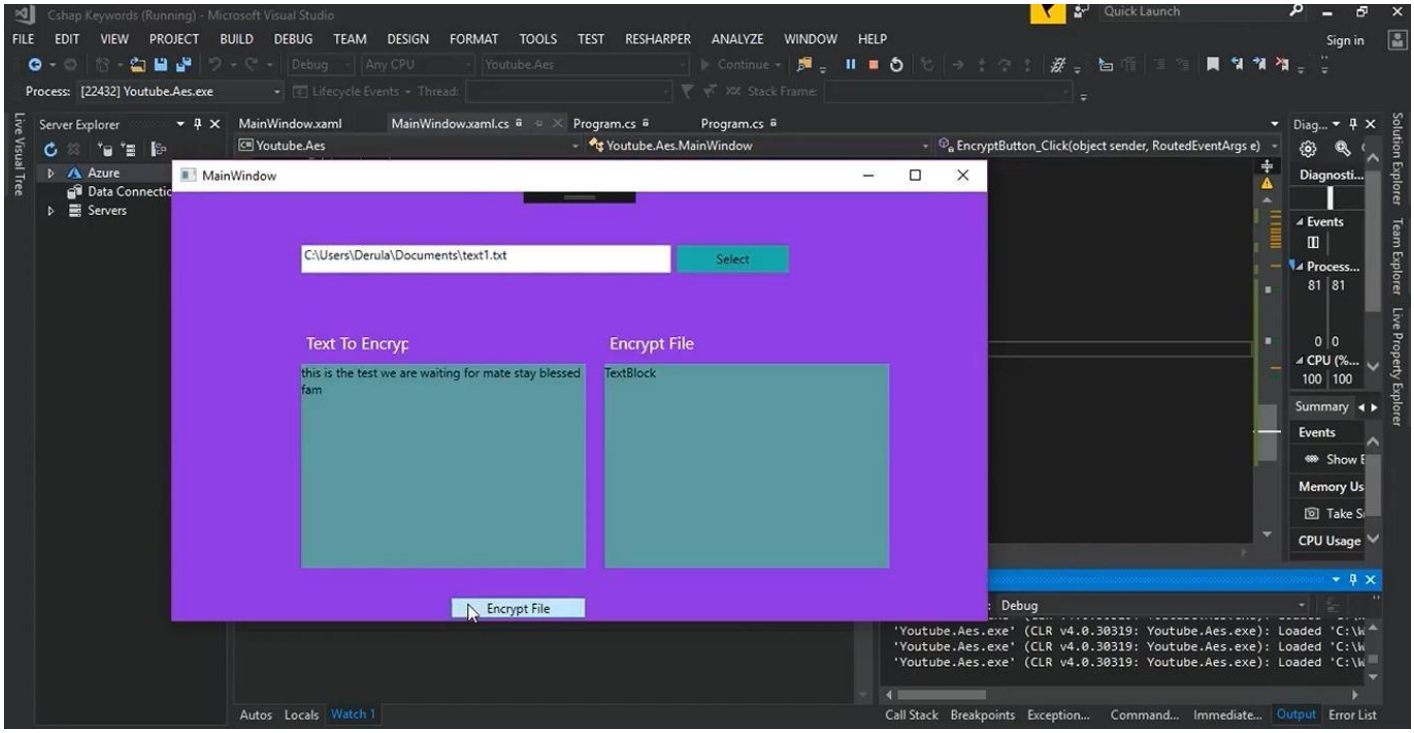
9.SAMPLE OUTPUT: TEXT TO ENCRYPT:



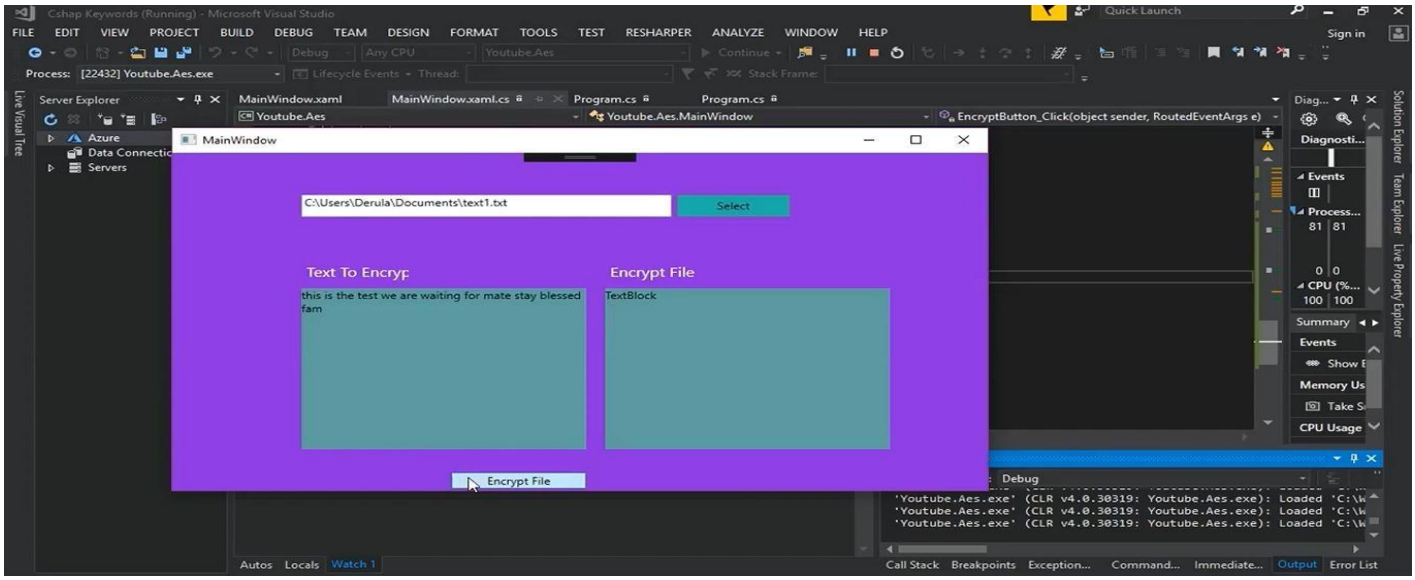
ENCRYTPT FILE:



DECRYPT AES ALGORITHM



DECRYPT FILE:



10. ANALYSIS:

1. Key & Parameter Generation

- A cryptographically secure pseudorandom number generator (CSPRNG) produces a unique **256-bit Key** and **12-byte Nonce**.
- **Critical Security Rule:** Never reuse the same (Key, Nonce) pair across multiple files, as doing so breaks GCM security and exposes plaintext.

2. Encryption Phase

- The file's raw bytes (Plaintext) are processed in 128-bit blocks through multiple rounds (14 rounds for AES-256) of substitution, permutation, and mixing steps.
- GCM mode simultaneously calculates an **Authentication Tag** alongside the encrypted bytes.
- The output file size increases slightly to accommodate the appended Auth Tag or overhead.

3. Decryption Phase

- The cipher takes the **Key**, **Nonce**, and **Encrypted File**.
- GCM mode verifies the **Auth Tag** *before* outputting plaintext. If even a single bit of the file was modified or corrupted, decryption immediately halts and fails (preventing active tampering).

4. Verification (SHA-256)

- Calculating the SHA-256 hash of the original file versus the decrypted file proves data integrity. Identical hash digests guarantee **zero data loss** or corruption.

5. Confidentiality

- AES-256 ensures that only users with the correct secret key can access the file.
- Unauthorized users cannot understand the encrypted data, even if they obtain the encrypted file.

6. Data Integrity

- The Authentication Tag generated by AES-GCM verifies that the encrypted file has not been altered.
- Any modification to the ciphertext results in decryption failure.

7. Authentication

- AES-GCM authenticates both the encrypted data and the encryption parameters.
- This prevents attackers from injecting or modifying data without detection.

8. Performance Analysis

- AES is one of the fastest symmetric encryption algorithms.
- It provides high-speed encryption and decryption with low computational overhead.
- Performance remains efficient even for large files.

11.**CONCLUSION:**

The 30-day internship at Fantasy Solution offered a thorough, structured learning path — from core Java fundamentals to advanced AES ALGORITHM, Hibernate, and Security concepts.

Applying these concepts to build the File Encryption and Decryption of AES algorithm made the learning practical and outcome-driven: the resulting application successfully digitizes event creation, registration, booking, payment, refund, and support processes for Admin, Organizer, and Attendee roles.

This experience has strengthened the understanding of full-stack Java development and has better prepared the intern for future software development roles and challenges.

12. FUTURE WORKS

- Integration of **AES-256 with RSA** for secure encryption key exchange (Hybrid Encryption).
- Development of a **cloud-based encryption system** for secure file storage and sharing.
- Implementation of **Two-Factor Authentication (2FA)** using OTP or authenticator applications for enhanced security.
- Addition of **biometric authentication** (fingerprint or facial recognition) before file encryption and decryption.
- Support for **multiple encryption algorithms** such as RSA, Blowfish, ChaCha20, and ECC, allowing users to choose the preferred encryption method.
- Development of **desktop, web, and mobile applications** to enable secure file access across different platforms.
- Integration of **digital signatures** to verify file authenticity and ensure non-repudiation.
- Automatic **secure key management** using encrypted key vaults or Hardware Security Modules (HSMs).
- Support for **batch encryption and decryption** to process multiple files simultaneously.
- Implementation of **secure file sharing** with password-protected encrypted links and expiry dates.
- Addition of **role-based access control (RBAC)** to restrict file access based on user roles and permissions.
- Integration with **cloud platforms** such as Google Drive, Dropbox, and Microsoft OneDrive for encrypted file synchronization.
- Implementation of **real-time audit logs** to monitor encryption, decryption, and file access activities.
- Support for **AI-based anomaly detection** to identify suspicious file access and potential cyber threats.
- Addition of **automatic backup and recovery** features for encrypted files.

13. REFERENCES

1. National Institute of Standards and Technology (NIST). FIPS PUB 197: Advanced Encryption Standard (AES). U.S. Department of Commerce, Gaithersburg, MD, USA, November 2001.
2. Dworkin, M. NIST Special Publication 800-38D: Recommendation for Block Cipher Modes of Operation – Galois/Counter Mode (GCM) and GMAC. National Institute of Standards and Technology, November 2007.
3. McGrew, D., RFC 5116: An Interface and Algorithms for Authenticated Encryption. Internet Engineering Task Force (IETF), January 2008.
4. Stallings, W. Cryptography and Network Security: Principles and Practice, 8th Edition, Pearson Education, 2020.
5. Paar, C., and Pelzl, J. Understanding Cryptography: A Textbook for Students and Practitioners. Springer, 2010.
6. Schneier, B. Applied Cryptography: Protocols, Algorithms, and Source Code in C, 2nd Edition, John Wiley & Sons, 1996.
7. Oracle Corporation. Java Cryptography Architecture (JCA) Reference Guide. Oracle Documentation.
8. Oracle Corporation. Java Cryptography Extension (JCE) Documentation. Oracle Java Security Documentation.
9. National Institute of Standards and Technology (NIST). SHA-256 Secure Hash Standard (SHS), FIPS PUB 180-4. U.S. Department of Commerce, 2015.
10. Menezes, A. J., van Oorschot, P. C., and Vanstone, S. A. Handbook of Applied Cryptography. CRC Press, 1996.